PROJECT PROGRESS — Dual-Mode Vision-Guided Robotic Arm

> Last updated: March 9, 2026 (Session 4)

Overall Status

Phase	Name	Status	Completion
Phase 0	URDF & Simulation	✅ Complete	100%
Phase 0b	CAD-Engineer URDF Migration	✅ Complete	100%
Pre	MediaPipe Detection	✅ Complete	100%
Demo	gesture_arm_demo — Live Hand Control	✅ Working	100%
Phase 1	MoveIt2 Setup	🔄 In Progress	40%
Phase 2	Gesture → Arm Control (full)	🔄 In Progress	50%
Phase 3	YOLOv8 Sorting	⏳ Pending	0%
Phase 4	MQTT Remote Control	⏳ Pending	0%
Phase 5	Hardware Deployment	⏳ Pending	0%

Phase 0 — URDF & Simulation Environment ✅

Status: COMPLETE

What was done

- [x] Received physical 6-DOF arm hardware
- [x] Sourced matching SolidWorks CAD model from community
- [x] Converted SolidWorks → Onshape (browser CAD) → STL via API
- [x] Obtained 40-link URDF from community model
- [x] Merged 40 links → 12 links using Python/NumPy cumulative transforms
- [x] All 43 STL meshes correctly placed and validated
- [x] `check_urdf` passes cleanly
- [x] Fixed Physics Error Code 19 (zero inertia → 1e-7 minimum)
- [x] Fixed robot invisible in Gazebo (set `GZ_SIM_RESOURCE_PATH` in launch file)
- [x] Fixed unsupported Ogre material strings (→ RGBA `<ambient>/<diffuse>`)
- [x] Fixed gripper continuously rotating (→ `revolute` joints ±0.5 rad limits)
- [x] Fixed RViz2 dropping all `/joint_states` (→ `joint_state_relay.py` node)
- [x] Wrote `scripts/joint_state_relay.py` (stamps `frame_id='base_link'`)
- [x] `launch/sim.launch.py` working — Gazebo + robot spawns
- [x] `launch/display.launch.py` working — RViz2 + joint sliders
- [x] Full TF tree verified in RViz2

PROF

Key files produced

- `src/robot_arm_description/urdf/robot.urdf.xacro`
- `src/robot_arm_description/urdf/gazebo.xacro`
- `src/robot_arm_description/config/bridge.yaml`
- `src/robot_arm_description/scripts/joint_state_relay.py`
- `src/robot_arm_description/launch/sim.launch.py`
- `src/robot_arm_description/launch/display.launch.py`

9 Simulation bugs resolved

#	Bug	Fix
1	Physics Error Code 19 — zero inertia	`<inertia>` all values → 1e-7 minimum
2	Robot invisible — mesh URI failed	`GZ_SIM_RESOURCE_PATH` in launch
3	Ogre material string unsupported	`<ambient>/<diffuse>` RGBA blocks
4	Gripper continuously rotating	`revolute` joints with ±0.5 rad limits
5	RViz2 drops all joint states	`joint_state_relay.py` stamps `frame_id`
6–9	(Additional physics / spawn issues)	See `docs/troubleshoots/`

Phase 0b — CAD-Engineer URDF Migration to Gazebo Harmonic ✅

Status: COMPLETE

Context

A CAD engineer provided a Fusion 360 → URDF exported model (`robotic_arm_description`).
The export targeted Gazebo Classic (ROS1) and was incompatible with our ROS2 Jazzy + Gazebo Harmonic stack.
This phase documents every issue encountered and the exact fix applied.

7 Issues Resolved

#	Issue	Symptom	Root Cause	Fix Applied
1	**All joints `continuous` with no limits**	Joints spin freely under gravity; arm collapses	Fusion exporter defaults to `continuous` type	Changed all 10 joi `<limit>` (lower/u and `<dynamics friction="0.1`
2	**4 `<mimic>` tags on gripper joints**	Gazebo Harmonic (DARTsim) ignores `<mimic>` → joints uncontrolled	DART physics engine has no mimic implementation	Removed all `<mim` gripper joint inde
3	**`libgazebo_ros_control.so` — Gazebo Classic plugin**	`[Err] Failed to load system plugin` → no controller manager	Plugin only exists in Gazebo Classic, not Harmonic	Removed plugin e
4	**`Gazebo/Silver` Ogre material strings**	Materials fail to load in Harmonic's rendering engine	Ogre material database removed in new Gazebo	Replaced with `<ambient>/<di` RGBA blocks
5	**`gz_ros2_control` ABI mismatch**	`undefined symbol: _ZTIN18hardware_interface26HardwareComponentInterfaceE` → plugin load fails → controller spawners retry forever → no `/joint_states`	apt-packaged `libgz_ros2_control-system.so` compiled against different `hardware_interface` version	Removed `gz_ros` bridged joint stat `ros_gz_bridge`
6	**No `world` link — robot tilts/falls**	Arm not fixed to ground, collapses under gravity	Fusion exporter doesn't add a `world → base_link` fixed joint	Added `<link na` `<joint name="` `type="fixed">`
7	**Gazebo not publishing joint states**	Bridge topic `/world/.../joint_state` exists but never receives data → RViz shows "No transform" for all links	Gazebo Harmonic does NOT auto-publish joint states (unlike Classic)	Added `gz-sim-j` `publisher-sys` `<gazebo>` block

Additional debugging notes

- **World name confusion:** `empty.sdf` creates a Gazebo world named `empty`, not `default` — verified with `gz topic -l | grep joint` and `gz topic -i`. The bridge must use `/world/empty/model/robotic_arm/joint_state`.
- **ROS1-style `<transmission>` tags:** The `.trans` file had `transmission_interface/SimpleTransmission` and `hardware_interface/EffortJointInterface` — these are ROS1 concepts. Replaced the entire file with a `<ros2_control>` block (kept for future `gz_ros2_control` from-source build).
- **KDL parser warning:** "root link base_link has an inertia" — resolved by the `world` dummy link (no inertia → KDL happy).

PROF

Files modified

File	Change
`urdf/robotic_arm.xacro`	All joints → `revolute` + limits + damping; removed `<mimic>`; added `world` link + fixed joint
`urdf/robotic_arm.gazebo`	Removed Classic plugin; added `JointStatePublisher` system plugin; `Gazebo/Silver` → RGBA materials
`urdf/robotic_arm.trans`	Replaced all `<transmission>` blocks with `<ros2_control>` hardware interface
`launch/sim.launch.py`	New ROS2 Python launch: Gazebo Harmonic + `ros_gz_bridge` (clock + joint states) + RSP + RViz2
`config/ros2_controllers.yaml`	New: controller definitions (for future `gz_ros2_control` from-source)
`package.xml`	Added runtime deps: `ros_gz_sim`, `ros_gz_bridge`, `xacro`, `rviz2`, etc.

Verification

- `check_urdf` passes cleanly (root: `world` → `base_link` → 10 child links)
- `colcon build` succeeds
- `ros2 launch robotic_arm_description sim.launch.py` spawns robot upright in Gazebo
- `/joint_states` publishes all 11 joints at sim rate

- RViz2 shows full TF tree with all links green

Pre-Phase — MediaPipe Hand Detection ✅

Status: VERIFIED WORKING

What was done

- [x] MediaPipe 0.10+ installed
- [x] Hand detection pipeline functional from laptop webcam
- [x] 21 landmarks extracted per hand at 30+ FPS
- [x] Landmark coordinate normalisation relative to wrist anchor tested

What remains (→ Phase 2)

- [ ] Landmark-to-joint-angle mapping logic
- [ ] Publish as `JointTrajectory` to `/arm_controller`
- [ ] Live arm mirroring test in Gazebo simulation

Phase 1 — MoveIt2 Setup 🔄 IN PROGRESS

Status: IN PROGRESS — MoveIt2 installed, `demo.launch.py` running, Setup Assistant config in progress

Tasks to complete

- [x] Install MoveIt2:

```
sudo apt install ros-jazzy-moveit
```

- [x] Created `src/robotic_arm_moveit_config/` package directory
- [x] `ros2 launch robotic_arm_moveit_config demo.launch.py` — runs successfully
- [ ] Launch MoveIt2 Setup Assistant:

```
ros2 launch moveit_setup_assistant setup_assistant.launch.py
```

- [ ] Load `robotic_arm_description` URDF in Setup Assistant
- [ ] Generate self-collision matrix (takes ~2 min)
- [ ] Add planning group: `arm` — joints `link_1` through `link_5`
- [ ] Add end-effector: `gripper` — joints `link_6` (+ follower joints `link_10`–`link_13`)
- [ ] Set home position: all joints at 0.0 rad
- [ ] Select kinematics solver: KDL (default) or TRAC-IK
- [ ] Configure `ros2_control` interface (position controllers)
- [ ] Generate and export `robotic_arm_moveit_config` package
- [ ] Build the new package: `colcon build`
- [ ] Test: drag interactive marker in RViz2 → Plan → Execute
- [ ] Verify arm moves collision-free in Gazebo simulation

PROF

Joint reference (from actual URDF)

Joint	Type	Links	Role
`link_1`	revolute	base_link → link_1_1	Base rotation
`link_2`	revolute	link_1_1 → link_2_1	Shoulder
`link_3`	revolute	link_2_1 → link_3_1	Elbow
`link_4`	revolute	link_3_1 → link_4_1	Wrist pitch
`link_5`	revolute	link_4_1 → link_5_1	Wrist roll
`link_6`	revolute	link_5_1 → Gripper_Link_Gear#1_v1_1	Gripper actuator
`link_10`–`link_13`	revolute	Gripper fingers	Gripper followers (ex-mimic)

Expected output

- New package: `src/robot_arm_moveit_config/`
- SRDF file with planning groups and collision matrix
- Working IK planning via RViz2 interactive markers

Demo — gesture_arm_demo: Live Hand → Arm Control ✅

Status: WORKING — standalone demo, no MoveIt2 required

What was built (Session 4)

A completely self-contained ROS2 package `gesture_arm_demo` that lets the arm mirror hand movements in real-time in RViz2. No Gazebo, no MoveIt2 needed — works right now.

Architecture

```
Webcam (OpenCV)
    ↓
MediaPipe HandLandmarker (tasks API, hand_landmarker.task model)
    ↓  21 NormalizedLandmark points (x,y ∈ [0,1])
_compute_joints()  — maps landmarks → 6 joint angles
    ↓
EMA filter (α=0.3)  — smooths jitter
    ↓
sensor_msgs/JointState → /joint_states  (30 Hz)
    ↓
robot_state_publisher  — broadcasts TF for all links
    ↓
RViz2  — renders robot model following your hand
```

Hand → Joint mapping (live)

Hand movement	Joint	URDF limits
Hand left ↔ right	`link_1` base rotation	±π
Hand up ↕ down	`link_2` shoulder	±π/2
Palm lean (wrist→knuckle angle)	`link_3` elbow	±π/2
Index finger point direction	`link_4` wrist pitch	±π/2
Knuckle-row tilt (roll)	`link_5` wrist roll	±π/2
Thumb–index pinch distance	`link_6` + followers gripper	±0.5 rad

Files produced

File	Purpose
`src/gesture_arm_demo/gesture_arm_demo/gesture_node.py`	Main node: webcam → landmarks → joint states
`src/gesture_arm_demo/launch/demo.launch.py`	Launches RSP + gesture_node + RViz2
`src/gesture_arm_demo/launch/gesture_demo.rviz`	Clean RViz2 config (ROS2 plugin names)
`src/gesture_arm_demo/setup.py`	Package setup
`src/gesture_arm_demo/package.xml`	Package manifest

PROF

Bugs fixed during session

#	Bug	Fix
1	`AttributeError: module 'mediapipe' has no attribute 'solutions'`	Switched to `mediapipe.tasks` API — same as working `hand_landmarker.py`
2	RViz2 errors: `rviz/Grid`, `rviz/RobotModel`, `rviz/Orbit` not found	Old `urdf.rviz` used ROS1 plugin names — created `gesture_demo.rviz` with correct `rviz_default_plugins/*` names

How to run

```
source /opt/ros/jazzy/setup.bash
source /home/natraj/DDS/install/setup.bash
ros2 launch gesture_arm_demo demo.launch.py
```

Phase 2 — Gesture → Arm Live Control (Full Integration) 🔄

Status: IN PROGRESS (50%) — core gesture logic done in demo, MoveIt2 integration pending

Done

- [x] MediaPipe landmarks extracted (21 points, x/y/z)
- [x] Wrist anchor normalisation logic tested
- [x] Full landmark-to-joint mapping implemented (`gesture_node.py`)
  - Wrist X position → `link_1` (base rotation)
  - Wrist Y position → `link_2` (shoulder)
  - Wrist→knuckle angle → `link_3` (elbow)
  - Index MCP→TIP angle → `link_4` (wrist pitch)
  - Knuckle row tilt → `link_5` (wrist roll)
  - Thumb–index distance → `link_6` (gripper)
- [x] EMA filter (α = 0.3) applied to all joints
- [x] All outputs clamped to URDF joint limits
- [x] Live arm mirroring in RViz2 verified and working

To complete (after Phase 1 is done)

- [ ] Replace direct `/joint_states` publish with `JointTrajectory` → `/arm_controller/joint_trajectory`
- [ ] Set trajectory duration floor: 150 ms minimum
- [ ] Test: raise hand → arm follows in Gazebo simulation (with controllers active)
- [ ] Test: lower hand → AUTO mode resumes

Depends on

- Phase 1 (MoveIt2 + `/arm_controller` must exist first)

Phase 3 — YOLOv8 Autonomous Sorting ⏳

Status: NOT STARTED

Tasks to complete

- [ ] Install IP Webcam app on phone, note stream URL
- [ ] Write `camera_node.py` — ingest MJPEG stream:

```
cap = cv2.VideoCapture("http://<phone-ip>:8080/video")
```

- [ ] Print and use checkerboard for camera calibration:

```
ros2 run camera_calibration cameracalibrator
```

- [ ] Save `camera_intrinsics.yaml`
- [ ] Train or load YOLOv8 model (pre-trained `yolov8n.pt` to start)
- [ ] Write `vision_node.py`:
  - YOLOv8 inference on each frame (CUDA)
  - Extract 2D bounding box centre
  - Apply homography: pixel → 3D world XYZ
  - Publish `/detected_objects`
- [ ] Write `sorting_planner_node.py`:
  - Map object class → target bin location
  - Generate target pose → `geometry_msgs/PoseStamped`
- [ ] Write full pick-sort-return cycle:
  - Move to object → close gripper → move to bin → open gripper → home
- [ ] Test full autonomous loop in Gazebo with placeholder objects

Depends on

- Phase 1 (MoveIt2), Phase 2 (gesture mode stable)

Phase 4 — MQTT Remote Control ⏳

Status: NOT STARTED

Tasks to complete

- [ ] Create HiveMQ Cloud account — free tier (100 connections)
- [ ] Note broker URL, port (8883 SSL), username, password
- [ ] Install paho-mqtt:

PROF

```
pip install paho-mqtt
```

- [ ] Write `mqtt_bridge_node.py`:
  - Subscribe to `arm/joints` topic on HiveMQ
  - Deserialise JSON `{j1, j2, j3, j4, j5, j6}` → degrees → radians
  - Publish `JointTrajectory` → `/arm_controller`
  - Publish latency feedback back to phone
- [ ] Write Flask web app (`web_app/app.py` + `templates/index.html`):
  - Serve a single HTML page
  - Page activates front camera
  - MediaPipe.js processes landmarks in browser
  - Maps to joint angles (same formula as gesture_node.py)
  - Publishes JSON via MQTT.js to HiveMQ
  - Displays live latency counter
- [ ] Install and configure ngrok:

```
ngrok http 5000
```

- [ ] Test: open ngrok URL on a phone → arm moves in simulation
- [ ] Test: 3-second timeout → arm returns home → AUTO resumes

Depends on

- Phase 1 (MoveIt2), MQTT account created

Phase 5 — Hardware Deployment ⏳

Status: NOT STARTED — requires all software phases complete

Tasks to complete

Electronics wiring

- [ ] Mount PCA9685 on breadboard
- [ ] PSU barrel → DC Jack → PCA9685 V+ / GND (20-22 AWG, screw terminal)
- [ ] ESP8266 3.3V → PCA9685 VCC (logic only)
- [ ] ESP8266 GND → PCA9685 GND (common ground — mandatory)
- [ ] ESP8266 D1 (GPIO5) → PCA9685 SCL
- [ ] ESP8266 D2 (GPIO4) → PCA9685 SDA
- [ ] MPU6050 → same I2C bus (addr 0x68), VCC from ESP8266 3.3V
- [ ] Connect 6× MG996R servos to PCA9685 CH0–CH5
- [ ] Power on — verify PCA9685 green LED lit

ESP8266 firmware

PROF

- [ ] Set up PlatformIO or Arduino IDE
- [ ] Write firmware:
  - I2C master to PCA9685
  - Receive joint angles over serial from ROS2
  - Convert radians → PWM microseconds (500–2500 µs)
  - Write to PCA9685 channels
- [ ] Flash firmware via micro-USB cable
- [ ] Test: send known angle → verify servo moves to expected position

ros2_control hardware interface

- [ ] Write `esp8266_hardware_interface.cpp` (or Python wrapper):
  - Implement `SystemInterface` — `read()` and `write()` methods
  - Serial communication to ESP8266 at 115200 baud
- [ ] Update `robot.urdf.xacro` — change `<plugin>` from `gz_ros2_control` to `ros2_control`
- [ ] Test: same `JointTrajectory` commands that worked in simulation → real arm moves

MPU6050 integration

- [ ] Write `imu_node.py` — read accelerometer + gyroscope from ESP8266 I2C
- [ ] Compute orientation quaternion (complementary filter or Madgwick)
- [ ] Publish `/imu/data` → use for drift correction in wrist joint

Final validation

- ☐ All 3 modes working on real hardware
- ☐ Safety features tested (timeout, clamping, home recovery)
- ☐ Latency measured end-to-end (real hardware path)

Hardware Procurement Status

Item	Status	Source	Cost
6-DOF metal arm kit	✅ On hand	—	—
6× MG996R servo	✅ On hand	—	—
ESP8266 NodeMCU	✅ On hand	—	—
PCA9685 16-ch driver	✅ On hand	—	—
5V / 10A PSU	✅ On hand	—	—
1080p webcam	✅ On hand	—	—
DC Jack Female (screw terminal)	✅ Ordered	—	—
MPU6050 IMU	✅ Ordered	indianhobbycenter.com	₹199
400-pt breadboard	✅ Ordered	indianhobbycenter.com	₹55
40-pin Dupont jumper wires	✅ Ordered	Amazon / Robu.in	~₹100
ESP32-CAM + MB base board	✅ Bought (spare)	—	~₹600
ESP32S OV3660 camera module	✅ Bought (spare)	—	included
2 mm flathead screwdriver	🛒 To buy	Hardware store	~₹50
Micro-USB cable	✅ On hand (likely)	—	—
Phone stand / overhead clamp	🛒 To buy	Amazon	~₹200
Electrical tape / heat shrink	🛒 To buy	Hardware store	~₹50
Zip ties	🛒 To buy	Any store	~₹30

Software Environment

Tool	Version	Status
Ubuntu	24.04 LTS	✅ Running
ROS2	Jazzy Jalisco	✅ Installed
Gazebo	Harmonic	✅ Installed
MoveIt2	—	✅ Installed
Python	3.10+	✅
OpenCV	4.8+	✅
YOLOv8 (Ultralytics)	Latest	✅ Installed
PyTorch + CUDA 12.1	2.x	✅ Installed
MediaPipe	0.10+	✅ Installed
paho-mqtt	—	⏳ To install
Flask	—	⏳ To install
ngrok	—	⏳ To install

PROF

Key Architectural Decisions (Locked)

Decision	Choice	Rationale
Fixed overhead camera	User's phone (IP Webcam app)	Free, 1080p+, MJPEG, works with OpenCV
Remote operator camera	Any phone — browser only	MediaPipe.js, no app install needed
Local gesture camera	Laptop built-in webcam	Already present, ROS2 usb_cam support
Servo MCU	ESP8266 NodeMCU	Wi-Fi capable, replaces Arduino Mega

Decision	Choice	Rationale
Servo driver	PCA9685 16-ch	I2C, 16 channels, 5V PWM output
Motion planning	MoveIt2 + OMPL	Collision-aware IK, industry standard
Remote protocol	MQTT via HiveMQ	NAT-penetrating, free, phone-compatible
Simulation	Gazebo Harmonic	Official ROS2 Jazzy pairing
ESP32-CAM (OV3660, 2MP)	Spare — future replacement	Bought as backup for phone camera; not active in current architecture
Rejected: OV7670	Not used	Parallel bus, no ROS2 driver, VGA only
Rejected: Arduino Mega	Not used	USB tether, no Wi-Fi

Update this file after completing each task.